

Day7 实训手册：Feed流与热点榜单 — Redis ZSet

一、今日任务目标

目标	说明
掌握 ZSet	member + score 的有序集合，天然支持排行榜与时间线
理解 Feed 流架构模式	推/拉/推拉结合的读写扩散权衡
实现关注 Feed	推模式（写扩散）：发笔记时推送到每个粉丝的收件箱
实现热点榜单	热度公式实时累加，Top10 一次 ZREVRANGE 取出
产出对比数据	关注页、热榜接口与 MySQL 版本的 RT 对比
课堂实战	实现“热门话题”排行榜（第八节，课内启动、课下完成）

二、问题场景（先复现，再优化）

场景一：关注页

1

2

优化前：每次刷新关注页 = 子查询 t_follow + 联表 t_note + 排序
用户关注越多、笔记越多，查询越慢

场景二：热榜

1

2

3

优化前：每次请求都执行
ORDER BY (like_count + comment_count*5 + favorite_count*2) DESC
全表聚合计算，数据量越大越慢

先用 JMeter 对 GET /api/notes/follow、GET /api/notes/hot 各压一轮，记录优化前数据。

三、理论基础：ZSet 原理与 Feed 流架构

3.1 ZSet：带分数的有序集合（本课主角）

1

2

ZSet = 一组 (member, score) 对，按 score 自动排序，member 唯一
test-rank: alice:90 bob:75 carol:88 ← 按分数自动排好序，可随时插入/加分/取排名

能力	复杂度	对应本课场景
插入/更新分数 (ZADD)	$O(\log N)$	发布笔记推入粉丝收件箱
分数增减 (ZINCRBY)	$O(\log N)$	点赞/评论实时累加热度
按分数取区间 (ZRANGE/ZREVRANGE)	$O(\log N + M)$	Top10 热榜、Feed 分页
查排名 (ZRANK)、查分数 (ZSCORE)	$O(\log N)$	了解即可
删除成员 (ZREM)、计数 (ZCARD)	$O(\log N)$	了解即可
数区间成员数 (ZCOUNT)	$O(\log N)$	了解即可

底层原理 (了解)：ZSet 由**哈希表 + 跳表**组成 —— 哈希表保证 $O(1)$ 查分数，跳表保证 $O(\log N)$ 插入和按分数范围查询。

这就是排行榜能实时维护排序的原因。对比 MySQL：插入不排序，排序靠查询时 `ORDER BY` (每次全表计算)。

3.2 动手：亲手做一个排行榜 (上课必做)

进入容器：`docker exec -it xhs-redis redis-cli:`

```
1 # ① 插入三名选手的分数 (ZADD key score member)
2 ZADD demo:rank 90 alice 75 bob 88 carol
3
4 # ② 从高到低取前2名 (ZREVRANGE, -WITHSCORES 带分数)
5 ZREVRANGE demo:rank 0 1 WITHSCORES
6
7 # ③ alice 又得分 +5, 实时反超 (ZINCRBY) → 再查一次排名看变化
8 ZINCRBY demo:rank 5 alice
9 ZREVRANGE demo:rank 0 -1 WITHSCORES
10
11 # ④ 查单人分数与名次 (从0开始)
12 ZSCORE demo:rank bob
13 ZREVRANK demo:rank bob
14
15 # ⑤ 清理
16 DEL demo:rank
```

课堂讨论：为什么热榜用 `ZINCRBY` 实时累加，而不是每次请求时重新计算全部热度？
(把“排序成本”从读时均摊到写时：每次互动多一次 $O(\log N)$ ，换来读榜单一次命令返回。)

3.3 Feed 流三种架构模式 (设计权衡，重点)

Feed 流 (信息流) 是当前互联网产品的核心基础设施。简单说，它是一种**将动态更新的内容按特定顺序聚合，并持续推送给用户的数据组织形式**。

关注页要回答一个看似简单、实则昂贵的问题：“我关注的这些人，最近发了什么？”——难点在于这些笔记分散在**成百上千个作者**名下，还要按时间排好序。业界把“排序合并这份成本放在写的时候还是读的时候”称为**读扩散 / 写扩散**的权衡，由此衍生出三种经典模式。

先看总览，再逐个拆解思路、流程与伪代码：

模式	一句话原理	写成本	读成本	适用
推模式 (写扩散)	发布时就把笔记推给所有粉丝的收件箱	高 (粉丝越多越重)	极低 (只读自己收件箱)	普通用户 (★本课)
拉模式 (读扩散)	读取时实时拉取所有关注人的笔记再归并	低 (只写自己发件箱)	高 (关注越多越慢)	关注少、发布多; 大V
推拉结合	普通用户推、大V不推改读时拉	中	中	微博/小红书生产方案

3.3.1 推模式 (写扩散) — 本课落地

核心思路：把“读时合并排序”的成本**提前到写时一次性完成**。作者发布笔记的瞬间，系统就把这条笔记的 `id` 主动“推”进他每一个粉丝的收件箱（每个收件箱是一个 `ZSet`，`score` 存发布时间戳）。粉丝刷新关注页时，只需读自己的收件箱—因为 `score` 就是时间戳，一次 `ZREVRANGE` 拿到的天然就是排好序的时间线，完全不用现场合并。

数据结构：

```
1 | ZSet Key: feed:{userId}      # 每个用户一个收件箱
2 |   member = noteId
3 |   score  = 发布时间戳 (毫秒)  # 按 score 倒序 = 时间线
```

写流程 (发布笔记时)：

```
1 | publish(note, authorId):
2 |     noteId = 保存笔记到 MySQL
3 |     fans   = 查询 authorId 的所有粉丝ID
4 |     score  = 当前时间戳(ms)
5 |     for fanId in fans:
6 |         ZADD feed:{fanId} score noteId      # 逐个推进粉丝收件箱 (写扩散发生在这里)
```

读流程 (刷新关注页)：

```
1 | followFeed(userId, page, size):
2 |     ids   = ZREVRANGE feed:{userId} start end      # 按时间倒序分页取 noteId
3 |     notes = selectByIds(ids)                       # 批量取详情 (走 Day2 缓存), FIELD()
   |     保序
4 |     return notes
```

对应本课真实代码 (`code/day7/service/FeedService.java`)：

```

1 // 发布时把笔记推进每个粉丝的收件箱
2 public void pushNote(Long noteId, Long authorId) {
3     List<Long> fans = followMapper.selectFollowerIds(authorId); // 查作者的所有粉丝
4     if (fans == null || fans.isEmpty()) {
5         return;
6     }
7     double score = System.currentTimeMillis(); // score = 发布时间戳
8     for (Long fanId : fans) {
9         stringRedisTemplate.opsForZSet()
10             .add(RedisKeys.feed(fanId), noteId.toString(), score); // ZADD feed:
11     }
12 }

```

优点：读极快，一次 **ZREVRANGE** 搞定排序+分页；读逻辑与关注关系解耦。**代价：**写放大—粉丝越多，一次发布要写的收件箱越多；大V发布近乎灾难（见下方思考）；收件箱会无限膨胀，生产上需定期裁剪。**适用：**粉丝量可控的普通用户，正是本课的场景。

3.3.2 拉模式（读扩散）

核心思路：把成本反过来放到**读时**。作者发布时几乎什么都不做（最多写一下自己的“发件箱” outbox）；粉丝刷新关注页时，系统实时去“拉”取自己关注的每一个人的最新笔记，在内存里做多路归并排序。写很轻，读很重。

数据结构：

```

1 ZSet Key: outbox:{authorId} # 每个作者一个发件箱（存自己发的笔记）
2 member = noteId
3 score = 发布时间戳（毫秒）

```

读流程（刷新关注页）：

```

1 followFeed(userId, page, size):
2     followees = 查询 userId 关注的所有人
3     candidates = []
4     for authorId in followees:
5         candidates += ZREVRANGE outbox:{authorId} 0 K # 每人各拉最新 K 条
6     merged = 按 score(时间戳) 多路归并排序(candidates) # 归并成本随关注数上升
7     return 分页(merged)

```

优点：写成本极低（发布只写一次自己的发件箱）；没有收件箱膨胀问题；大V发布毫无压力。**代价：**读成本高—关注越多，要拉的人越多、归并越慢，延迟随关注数线性增长；深分页尤其困难（每翻一页都要重新归并）。**适用：**关注数少但发布频繁的用户，或作为大V的读取方式（见 3.3.3）。

本课不实现拉模式，但理解它的“读时归并”是理解推拉结合的前提。

3.3.3 推拉结合（生产方案）

核心思路：按“粉丝量”分流，取两者之长。**普通用户走推模式**（粉丝少，推送成本低，换来极佳的读体验）；**大V走拉模式**（粉丝上千万，推送不现实，改为粉丝读时主动拉大V的最新几条）。读取关注页时，把“收件箱里已被推来的普通关注人笔记”与“实时拉取的大V发件箱笔记”两部分**归并**成完整时间线。

大V判定：粉丝数超过阈值（如 1 万）即标记为大V，发布时不推、只写发件箱。

写流程：

```
1 publish(note, authorId):
2     if isBigV(authorId):
3         ZADD outbox:{authorId} score noteId           # 大V：只写自己发件箱，不扩散
4     else:
5         for fanId in fans:
6             ZADD feed:{fanId} score noteId           # 普通用户：照常推给粉丝
```

读流程：

```
1 followFeed(userId):
2     pushed = ZREVRANGE feed:{userId} ...             # 已推进收件箱的（普通关注人）
3     bigVs = 查询 userId 关注的大V列表
4     pulled = []
5     for v in bigVs:
6         pulled += ZREVRANGE outbox:{v} 0 K           # 实时拉每个大V的最新 K 条
7     merged = 按时间戳归并排序(pushed + pulled)
8     return 分页(merged)
```

优点：兼顾读写——普通用户读得快，大V写不炸，是微博/小红书等真实社交产品的实际选型。**代价：**实现最复杂，要额外维护大V名单、两套存储（收件箱 + 发件箱）以及读时归并逻辑。**适用：**用户粉丝量分布极不均匀（少数大V吸走绝大多数关注）的真实产品。

结论：本课用户量小、粉丝少 → 选最简单的推模式即可跑通关注页；拉模式与推拉结合只需理解其“读写成本互换”的思想。

思考题：如果作者有 1000 万粉丝，发一篇笔记要写 1000 万个收件箱怎么办？（异步化 + 只推活跃粉丝 + 大V改拉模式 —— 这就是推拉结合。）

另一个工程细节：收件箱不能无限增长，生产上会定期 `ZREMRANGEBYRANK feed:{id} 0 -101` 只保留最新 100 条（本课数据量小，不做裁剪）。

3.4 热度算法与榜单设计（了解即可，本项目用简化版）

本课热榜用的是最简单的**线性加权求和**：**热度 = 点赞×1 + 评论×5 + 收藏×2**，靠实时 `ZINCRBY` 累加。它能跑通“互动越多越靠前”，但有两个明显短板：① **只增不减**，早期爆款会长期霸榜，新内容很难冒头；② **只看互动绝对量**，不看新鲜度，也不防刷。生产级热榜会引入更精细的算法。本节把常用方案讲清楚——**本项目不实现，认知即可**，但面试与真实排榜设计里它们高频出现。

3.4.1 本项目的简化公式（基线）

```
1 | hot = like_count×1 + comment_count×5 + favorite_count×2
```

权重设计的直觉：**评论**成本最高、最能体现内容质量，权重最高（×5）；**收藏**代表“实用价值/以后还想看”，次之（×2）；**点赞**最轻量，权重最低（×1）。落地方式对应真实代码：**HotService** 里三个权重常量 + **InteractService** 在点赞/收藏成功时 **ZINCRBY**（取消时累加负值）+ **InitHotRunner** 冷启动用存量 count 重建，MySQL 兜底公式见 **NoteMapper.selectHot** 的 **ORDER BY (like_count + comment_count×5 + favorite_count×2)**。

短板：没有时间因子，热度只增不减 → 老帖霸榜。下面的算法基本都在解决“新鲜度”和“抗刷”这两件事。

3.4.2 五种常用热度算法对比

算法	核心公式	时间衰减	抗刷/样本修正	典型适用榜单
线性加权（★本课）	$\Sigma(\text{互动次数} \times \text{权重})$	无	靠幂等去重	数据量小的简单热榜
Hacker News	$(\text{票数} - 1) \div (\text{年龄小时} + 2)^{1.8}$	有（幂律衰减）	一般	资讯/新内容优先榜
Reddit Hot	$\log_{10}(\text{票数}) + \text{方向} \times \text{秒数} \div 45000$	有（线性时间项）	对数抑制刷票	综合热度社区榜
牛顿冷却	$\text{初始热度} \times e^{(-\lambda \times \text{经过时间})}$	有（指数衰减）	一般	时效性强、快速降温榜
威尔逊区间	好评率的置信下界	无	按样本量修正	好评率/内容质量榜

3.4.3 Hacker News 算法（时间衰减的经典）

```
1 | score = (votes - 1) / (age_hours + 2) ^ gravity # gravity 默认 1.8
```

分子是净票数，分母是“发帖至今的小时数 + 2”的 **gravity** 次幂。帖子越老，分母越大，得分呈幂律下降——于是新内容有机会冲榜，旧内容自然沉底。**gravity** 越大衰减越快（调节“喜新厌旧”的程度）。它只需要票数和发帖时间两个输入，简单高效，是“新内容优先”的资讯榜的经典选择。

3.4.4 Reddit Hot 算法（对数 + 时间偏移）

```
1 | z      = 净投票数 (赞 - 踩)
2 | order  = log10( max(|z|, 1) ) # 票数的“量级”
3 | seconds = 发帖时间 - 基准纪元 (如 2005-12-08)
4 | hot    = order + sign(z) × seconds / 45000
```

两部分相加：**log10(票数)** 让票数增长**边际递减**——前 10 票带来的提升远大于后 100 票，天然抑制刷票；**时间 ÷ 45000** 给新帖一个基础加分（45000 秒 ≈ 12.5 小时，相当于“每新 12.5 小时 ≈ 多一个数量级的票”）。它同时兼顾热度与新鲜度，是综合社区榜的代表算法。

3.4.5 牛顿冷却定律（指数时间衰减）

```
1 | 当前热度 = 初始热度 × e(-λ × 经过时间)           # λ 为冷却系数
```

模拟“热物体自然冷却”：内容热度随时间指数衰减，无人互动就快速降温。工程上常配合**定时批量重算**（例如每小时给榜单整体乘一个衰减因子（如 $\times 0.9$ ），或按每个成员的“最后互动时间”重算。适合时效性极强、要求“越新越热”的榜单。

3.4.6 威尔逊区间（Wilson Score，按“比率”排名）

当排名依据是**好评率 / 点赞率**这类“比率”而非绝对量时会遇到陷阱：直接用 $\text{赞} \div (\text{赞} + \text{踩})$ ，会让“1 赞 0 踩 = 100%”排在“1000 赞 10 踩 = 99%”前面，显然不合理（样本太小，比率不可信）。威尔逊区间给出“在给定置信度下，真实比率的**置信下界**”：样本越大、比率越高，下界越高，从而同时兼顾“比率高低”与“样本多少”。电商好评榜、内容质量榜常用；因计算较复杂，通常离线算好后写入 ZSet 的 score。

3.4.7 工程落地要点（榜单设计的其它考量）

- **计算时机**：实时累加（本课，互动频繁但基数小，**ZINCRBY** 即时生效） vs 定时批量重算（每小时跑批，适合带时间衰减的复杂公式与大规模数据）。
- **时间分桶榜单**：小时榜 / 日榜 / 周榜——用带时间后缀的 Key（如 **hot:notes:20260907**）分别累加，读取时按需选取或合并；老桶设 TTL 自动过期，既省内存又能做“历史榜”。
- **冷启动 / 重建**：榜单为空或 Redis 重启后，用 MySQL 存量数据重建（**InitHotRunner** 的思路）——这正是降级与“数据源永远在 DB”思想的又一次体现（Day8 深化）。
- **防刷**：同一用户重复互动要去重（Day3/Day4 的 Set 幂等已解决），对异常增长的热度还要限流兜底（Day8）。

一句话收束：本课的线性加权是“能跑通”的最小可用版；真实产品会叠加**时间衰减**（HN/Reddit/牛顿冷却）与**质量修正**（威尔逊区间），再配合定时重算、分桶榜单和防刷，才是完整的热榜设计。第八节“话题榜时间衰减”的选做挑战，用的就是这里的时间权重思想。

四、方案设计

4.1 关注 Feed（推模式）

```
1 | ZSet Key: feed:{userId}
2 |   member = noteId
3 |   score  = 发布时间戳（毫秒）
4 |
5 | 发布笔记时：
6 |   查出作者的所有粉丝 → 逐个 ZADD feed:{粉丝} noteId 当前时间戳
7 |
8 | 读取关注页时：
9 |   ZREVRANGE feed:{userId} 分页 → 批量取笔记详情（走Day2的缓存）
```

4.2 热点榜单

```
1 ZSet Key: hot:notes
2     member = noteId
3     score  = 热度值
4
5 热度公式: 点赞×1 + 评论×5 + 收藏×2
6 触发点:   点赞成功 → ZINCRBY +1
7           收藏成功 → ZINCRBY +2
8           评论成功 → ZINCRBY +5
9
10 读热榜: ZREVRANGE hot:notes 0 9 WITHSCORES → 组装笔记信息
11 冷启动: 榜单为空时回退 MySQL 计算 (降级思想, Day8 会深化)
```

五、编码实现

按 [code/day7/](#) 目录完成。先看改动全景与调用关系，再逐个类拆解职责与关键实现。

5.0 改动全景与调用关系

序号	文件	操作	核心方法 / 改动点
1	mapper/FollowMapper.java	【替换】	新增 <code>selectFollowerIds(userId)</code> 查粉丝ID
2	service/FeedService.java	【新增】	<code>pushNote</code> 写扩散推送、 <code>feedIds</code> 分页读收件箱
3	service/HotService.java	【新增】	<code>addHeat</code> 热度累加、 <code>topIds</code> 取 Top N、 <code>isEmpty</code> 冷启动判断
4	service/InitHotRunner.java	【新增】	<code>run()</code> : 榜单为空时用存量数据重建
5	service/InteractService.java	【替换】	点赞/收藏 (含取消) 成功后 <code>addHeat</code>
6	service/CommentService.java	【替换】	评论成功后 <code>addHeat</code> (+5)
7	service/NoteService.java	【替换】	<code>followFeed/hot</code> 改走 Redis、 <code>publish</code> 推送 Feed

```

1  【写链路】
2  publish(发笔记) → FeedService.pushNote → FollowMapper.selectFollowerIds → ZADD
   feed:{每个粉丝}
3  like/favorite/comment(互动) → HotService.addHeat → ZINCRBY hot:notes
4
5  【读链路】
6  followFeed(关注页) → FeedService.feedIds(ZREVRANGE) → NoteMapper.selectByIds → 空
   则回退 selectFollowFeed
7  hot(热榜) → HotService.topIds(ZREVRANGE) → NoteMapper.selectByIds → 空则回退
   selectHot
8
9  【启动】
10 InitHotRunner.run → hotService.isEmpty? → 否则用 MySQL 存量 count 重建 hot:notes

```

一句话把七个类串起来：**写的时候**把笔记推进粉丝收件箱、把互动累加进热度榜；**读的时候**直接从两个 ZSet 取排好序的 ID 再批量回填详情；**启动的时候**若榜单为空就用 MySQL 存量兜底重建。

5.1 FollowMapper: 新增“查粉丝 ID 列表”

职责：推模式写扩散时，需要知道“谁关注了作者”。新增一个只返回粉丝 ID 的轻量查询（不联表，只要 id）：

```

1  /** ★ Day7 新增：查询某用户的所有粉丝ID (Feed 推模式用) */
2  @Select("SELECT user_id FROM t_follow WHERE follow_user_id = #{userId}")
3  List<Long> selectFollowerIds(@Param("userId") Long userId);

```

5.2 FeedService (新增)：Feed 推送与读取

职责：封装推模式的两个动作——发布时写扩散、读取时分页拉收件箱。Key `feed:{userId}`，member=noteId，score=发布时间戳(ms)。

```

1  /** 发布笔记后调用：推给作者的每个粉丝 (写扩散) */
2  public void pushNote(Long noteId, Long authorId) {
3      List<Long> fans = followMapper.selectFollowerIds(authorId);
4      if (fans == null || fans.isEmpty()) {
5          return;
6      }
7      double score = System.currentTimeMillis();           // score = 发布时间戳
   (ms)
8      for (Long fanId : fans) {
9          stringRedisTemplate.opsForZSet()
10             .add(RedisKeys.feed(fanId), noteId.toString(), score); // ZADD
11     }
12 }
13
14 /** 分页读取某用户的收件箱 (最新在前)，返回笔记ID列表 */
15 public List<Long> feedIds(Long userId, int page, int size) {
16     long start = (long) (page - 1) * size;
17     long end = start + size - 1;
18     Set<String> members = stringRedisTemplate.opsForZSet()
19         .reverseRange(RedisKeys.feed(userId), start, end); // ZREVRANGE

```

```

20 // ..... 把 member 字符串解析为 Long 返回 (空则返回 emptyList)
21 }

```

要点：用 `StringRedisTemplate` (member/score 都是字符串)；score 用毫秒时间戳，`ZREVRANGE` 倒序即时线。

5.3 HotService (新增)：热度累加与 Top N

职责：维护热度榜 ZSet `hot:notes` (member=noteId, score=热度)，对外只暴露“加分/取榜/判空”三个动作，把权重集中管理。

```

1  /** 热度权重 */
2  public static final double WEIGHT_LIKE = 1;
3  public static final double WEIGHT_FAVORITE = 2;
4  public static final double WEIGHT_COMMENT = 5;
5
6  /** 累加热度 (可为负, 用于取消操作) */
7  public void addHeat(Long noteId, double weight) {
8      stringRedisTemplate.opsForZSet()
9          .incrementScore(RedisKeys.HOT_NOTES, noteId.toString(), weight); //
10     ZINCRBY
11 }
12
13 /** 热度前 N 的笔记ID (score 倒序) */
14 public List<Long> topIds(int limit) {
15     Set<String> members = stringRedisTemplate.opsForZSet()
16         .reverseRange(RedisKeys.HOT_NOTES, 0, limit - 1); //
17     ZREVRANGE 0 N-1
18     // ..... 解析为 Long 返回
19 }
20
21 /** 榜单是否为空 (冷启动判断) */
22 public boolean isEmpty() {
23     Long size = stringRedisTemplate.opsForZSet().zCard(RedisKeys.HOT_NOTES); //
24     ZCARD
25     return size == null || size == 0;
26 }

```

要点：`addHeat` 的 `weight` 可为负，取消点赞/收藏时传入 `-WEIGHT_*` 即可回退热度，一个方法搞定加减。

5.4 InitHotRunner (新增)：热榜冷启动

职责：实现 `CommandLineRunner`，启动时若 `hot:notes` 为空 (首次上线或 Redis 重启)，用 MySQL 存量 count 按同一套权重重建一次。

```

1  @Override
2  public void run(String... args) {
3      if (!hotService.isEmpty()) {
4          return; // 榜单已有数据, 不重建
5      }
6      List<Note> notes = noteMapper.selectList(null);
7      for (Note note : notes) {

```

```

8         double heat = note.getLikeCount() * HotService.WEIGHT_LIKE
9             + note.getCommentCount() * HotService.WEIGHT_COMMENT
10            + note.getFavoriteCount() * HotService.WEIGHT_FAVORITE;
11        if (heat > 0) {
12            hotService.addHeat(note.getId(), heat);
13        }
14    }
15    log.info("热榜冷启动初始化完成, 共 {} 篇笔记参与排序", notes.size());
16 }

```

要点：重建公式与 MySQL 兜底 `selectHot` 的 `ORDER BY` 一致；这也是“Redis 数据丢了就从 DB 重建”降级思想的体现（呼应 3.4.7）。

5.5 InteractService (替换)：互动成功后累加热度

职责：在 Day3/Day4 的幂等点赞/收藏逻辑上，四个方法各自插入一行 `addHeat`（其余 Lua、发消息逻辑不变）：

```

1 // 点赞成功 → +1 (LIKE_SCRIPT 幂等校验、发送落库消息等 Day3/Day4/Day6 逻辑保持不变)
2 hotService.addHeat(noteId, HotService.WEIGHT_LIKE);           // like
3 hotService.addHeat(noteId, -HotService.WEIGHT_LIKE);          // unlike (取消→负值)
4 hotService.addHeat(noteId, HotService.WEIGHT_FAVORITE);        // favorite
5 hotService.addHeat(noteId, -HotService.WEIGHT_FAVORITE);       // unfavorite

```

要点：热度累加只在幂等校验通过后执行（`result==1` 才加），避免重复点赞刷热度；取消操作对称地累加负值。

5.6 CommentService (替换)：评论成功后累加热度

职责：在 `add` 方法“插入评论 + 评论数+1 + 发通知消息”之后，追加一行热度 +5（权重最高的互动）：

```

1 // .....校验、插入评论、comment_count+1、发送 CommentEvent (Day6 逻辑保持不变)
2 // ★ Day7: 评论 → 热度 +5
3 hotService.addHeat(noteId, HotService.WEIGHT_COMMENT);
4 return Result.ok();

```

5.7 NoteService (替换)：关注页/热榜改走 Redis，发布推送 Feed

职责：把关注页与热榜的读取改为“先走 ZSet、空则回退 MySQL”，并在发布笔记后触发写扩散（详情缓存/分布式锁的 Day5 逻辑保持不变）：

```

1 /** ★ 关注页：走 ZSet Feed 收件箱，为空（新用户/冷启动）时回退 MySQL 联表 */
2 public List<NoteVO> followFeed(Long userId, int page, int size, Long viewerId) {
3     List<Long> ids = feedService.feedIds(userId, page, size);
4     List<NoteVO> list = ids.isEmpty()
5         ? noteMapper.selectFollowFeed(userId, (page - 1) * size, size) // 兜底
6         : noteMapper.selectByIds(ids);                                // 批量取
7     // 详情 (FIELD 保序)
8     fillStatus(list, viewerId);
9     mergeCounts(list);
10    return list;
11 }
12 /** ★ 热榜：走 ZSet 热度榜，为空（冷启动）时回退 MySQL 聚合 */

```

```
13 public List<NoteVO> hot(Long viewerId) {
14     List<NoteVO> list = hotService.isEmpty()
15         ? noteMapper.selectHot() // 兜底
16         : noteMapper.selectByIds(hotService.topIds(10)); // Top10
17     fillStatus(list, viewerId);
18     mergeCounts(list);
19     return list;
20 }
21
22 /** ★ 发布笔记后推送到所有粉丝的 Feed 收件箱 */
23 public Long publish(Note note, Long userId) {
24     // .....校验 + 初始化计数 + noteMapper.insert(note)
25     feedService.pushNote(note.getId(), userId); // 写扩散 (大V可改异步, 此处同步演示)
26     return note.getId();
27 }
```

要点: `selectByIds` 用 MySQL `FIELD()` 保留 ZSet 返回的顺序 (热度/时间序不会被重新排掉) ; 两处读取都保留了“ZSet 为空 → 回退 MySQL”的兜底, 保证冷启动/Redis 丢数据时接口不挂。

六、验证与压测

6.1 功能验证

步骤	操作	预期
1	启动后端	控制台打印热榜初始化日志
2	<code>ZRANGE hot:notes 0 -1 WITHSCORES</code>	12篇笔记按热度排序
3	登录 <code>beach_girl</code> , 发布一篇新笔记	-
4	登录她的粉丝账号, 查看关注页	新笔记出现在第一条
5	<code>ZRANGE feed:{粉丝id} 0 -1</code>	包含新笔记的noteId
6	给某笔记点赞5次	该笔记热度分 +5, 热榜排名变化

6.2 压测对比

接口	优化前RT	优化后RT	优化前QPS	优化后QPS
GET /notes/follow				
GET /notes/hot				

七、结果记录表

项目	结果
Feed 推送是否生效	
热榜排序是否符合公式	
关注页 RT 变化	
热榜 RT 变化	

八、课堂实战：热门话题排行榜（自己动手）

今天老师带做的是笔记热榜，现在轮到你用同一个武器做一个新榜单——**热门话题**。
小红书的标签云背后就是一个 ZSet。

需求：发布笔记时，把笔记的每个标签热度 +1；新增接口 `GET /api/notes/tags/hot` 返回 Top10 热门话题：

- Key: `hot:tags`，member 是标签文本（如 `三亚`），score 是出现次数；
- 发布笔记时：对 `tags` 逐个 `ZINCRBY hot:tags 1 {tag}`；
- 读榜单：`ZREVRANGE hot:tags 0 9 WITHSCORES`。

实现步骤：

1. `RedisKeys.java` 新增 `hotTags()` 返回 `"hot:tags"`；
2. 新建 `service/TagService.java`：`addHeat(List<String> tags)` 与 `topTags(int n)` 两个方法；
3. `NoteService.publish` 发布成功后调用 `tagService.addHeat(note.getTags())`（注意 `tags` 可能为空）；
4. `NoteController` 新增 `GET /tags/hot` 接口（注意路由写在 `/id` 之前，或确认不冲突）；
5. （选做）冷启动：启动时扫全表笔记的 `tags` 重建 `hot:tags`，仿照 `InitHotRunner` 的套路。

关键代码提示：

```
1 public void addHeat(List<String> tags) {
2     if (tags == null || tags.isEmpty()) return;
3     for (String tag : tags) {
4         redisTemplate.opsForZSet().incrementScore(RedisKeys.hotTags(), tag, 1);
5     }
6 }
7
8 public List<Map<String, Object>> topTags(int n) {
9     Set<ZSetOperations.TypedTuple<String>> tuples =
10         redisTemplate.opsForZSet().reverseRangeWithScores(RedisKeys.hotTags(), 0,
11             n - 1);
12     // 组装成 [{tag: "三亚", score: 12}, ...] 返回
13 }
```

验收标准：

- 发布一篇带 `三亚`、`环岛` 标签的笔记后，`ZSCORE hot:tags 三亚` 加 1；
- `GET /api/notes/tags/hot` 返回按热度降序的标签列表；

多发几篇不同标签的笔记，榜单顺序随发布实时变化；

`DEL hot:tags` 后榜单接口返回空列表不报错（降级思想）。

选做挑战：话题榜也会随时间过时，怎么给热度加“时间衰减”？（提示：回顾热榜方案里的时间权重设计）

九、思考题

1. 推模式的缺点是什么？大V（1000万粉丝）发笔记会发生什么？拉模式/推拉结合怎么解决？
2. 热度公式里评论为什么权重最高（×5）？
3. 如果 Redis 重启，feed 和 hot:notes 丢了怎么办？（提示：数据源仍在 MySQL，可重建 — `InitHotRunner` 就是重建思路）